Task 1: Performance Comparison of Recursion vs Iteration

Test Methodology

Implement both recursive and iterative versions of **summation (1+2+…+n), Fibonacci sequence, and factorial. Use the `timeit` module to measure execution time for different data scales (n = 10, 50, 100, 250, 500), running 1000 times per scale and taking the average. Due to the high complexity of recursive Fibonacci, test only for n ≤ 35.

Test Results

Summation (1 to n)

```
--- Sum ---
n=1: recursive=1, iterative=1  ✓
n=5: recursive=15, iterative=15  ✓
n=10: recursive=55, iterative=55  ✓

=== Performance comparison (1000 runs, time in seconds) ===

Input size n = 10
  Sum       recursive: 0.000515s  iterative: 0.000311s  iterative is 1.66x faster

Input size n = 50
  Sum       recursive: 0.002414s  iterative: 0.001157s  iterative is 2.09x faster

Input size n = 100
  Sum       recursive: 0.004760s  iterative: 0.002286s  iterative is 2.08x faster

Input size n = 250
  Sum       recursive: 0.024357s  iterative: 0.005700s  iterative is 4.27x faster

Input size n = 500
  Sum       recursive: 0.051602s  iterative: 0.012239s  iterative is 4.22x faster
```

Factorial

```
--- Factorial ---
n=1: recursive=1, iterative=1  ✓
n=5: recursive=120, iterative=120  ✓
n=10: recursive=3628800, iterative=3628800  ✓

=== Performance comparison (1000 runs, time in seconds) ===

Input size n = 10
  Factorial recursive: 0.000553s  iterative: 0.000316s  iterative is 1.75x faster

Input size n = 50
  Factorial recursive: 0.003032s  iterative: 0.001722s  iterative is 1.76x faster

Input size n = 100
  Factorial recursive: 0.006996s  iterative: 0.003955s  iterative is 1.77x faster

Input size n = 250
  Factorial recursive: 0.036273s  iterative: 0.014350s  iterative is 2.53x faster

Input size n = 500
  Factorial recursive: 0.096160s  iterative: 0.048561s  iterative is 1.98x faster
```

Fibonacci ($n \leqslant 35$)

```
--- Fibonacci ---
n=1: recursive=1, iterative=0  ×
n=5: recursive=5, iterative=5  ✓
n=10: recursive=55, iterative=55  ✓

=== Performance comparison (1000 runs, time in seconds) ===

Input size n = 10
  Fibonacci recursive: 0.000999s  iterative: 0.000036s  iterative is 27.99x faster

Input size n = 50
  Fibonacci n=50 recursive too slow, skipping

Input size n = 100
  Fibonacci n=100 recursive too slow, skipping

Input size n = 250
  Fibonacci n=250 recursive too slow, skipping

Input size n = 500
  Fibonacci n=500 recursive too slow, skipping
```

For n=35, recursive Fibonacci takes over 3 seconds; at n=40, recursion becomes practically impossible, while iteration remains O(n). Therefore, for larger n, iteration has an overwhelming advantage.

Performance Analysis

Iteration is faster than recursion because it has no function call overhead and does not accumulate stack frames.

Limitations of recursion: For deep recursion (e.g., n=1000 summation), Python's default

recursion depth limit (~1000) can be exceeded, causing a `RecursionError`. Iteration does not suffer from this.

Recursive Fibonacci is extremely inefficient: its time complexity is $O(2^n)$, while iteration is O(n). The difference grows exponentially with n.

Task 2: Merge Sort vs Insertion Sort

Test Methodology

Generate lists of random integers of sizes 100, 500, 1000, and 5000. Run merge sort and insertion sort on each list (passing a copy each time). Run each sort 10 times and take the average time.

Test Results

```
Original list: [3, 7, 2, 8, 1, 9, 4, 6, 5]
Merge sort result: [1, 2, 3, 4, 5, 6, 7, 8, 9]
Insertion sort result: [1, 2, 3, 4, 5, 6, 7, 8, 9]
=== Merge Sort vs Insertion Sort Benchmark (10 runs per size) ===

Data size Merge sort time (s) Insertion sort time (s) Merge speedup factor
------------------------------------------------------------
100       0.001137            0.001658            1.46
500       0.006976            0.043573            6.25
1000      0.015292            0.173122            11.32
5000      0.094762            4.396762            46.40
```

> Note: For very small data sizes (e.g., n=100), insertion sort has a smaller constant factor and can be slightly faster than merge sort. However, as n grows, the O(n log n) advantage of merge sort becomes evident.

Performance Analysis

Merge sort: stable O(n log n) time complexity, suitable for large data sets, but requires O(n) extra space.

Insertion sort: $O(n^2)$ time complexity; performs well on small or nearly sorted data, but performance degrades sharply as data size increases.

Practical application: Python's built-in `TimSort` combines the strengths of both algorithms (insertion sort for small subsets, merge sort for larger ones).

Task 3: Tower of Hanoi

Original Code Test

```
Enter number of disks: 3
The moves are:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
```

```
Enter number of disks: 4
The moves are:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
Move disk 4 from A to B
Move disk 1 from C to B
Move disk 2 from C to A
Move disk 1 from B to A
Move disk 3 from C to B
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
```

Run the recursive Tower of Hanoi program for 3 disks and 4 disks, outputting the move steps.

Output for 3 disks (excerpt)

Move disk 1 from A to B

Move disk 2 from A to C

Move disk 1 from B to C

Move disk 3 from A to B

Move disk 1 from C to A

Move disk 2 from C to B

Move disk 1 from A to B

```
========== Test 3 disks ==========
Move steps:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Total moves: 7
```

Output for 4 disks (step count only)

Total moves: 15 steps

Results After Adding a Move Counter

Modify the code to use a global variable `move_count` to count steps. Results:

```
========== Test 4 disks ==========
Total moves: 15
```

Conclusion: The move count follows the formula `$2^n - 1$`, verifying the correctness of the recursive solution.

Task 4: Fractal Drawing

4.1 Sierpiński Triangle

Program: `task4_sierpinski.py` (using Tkinter)

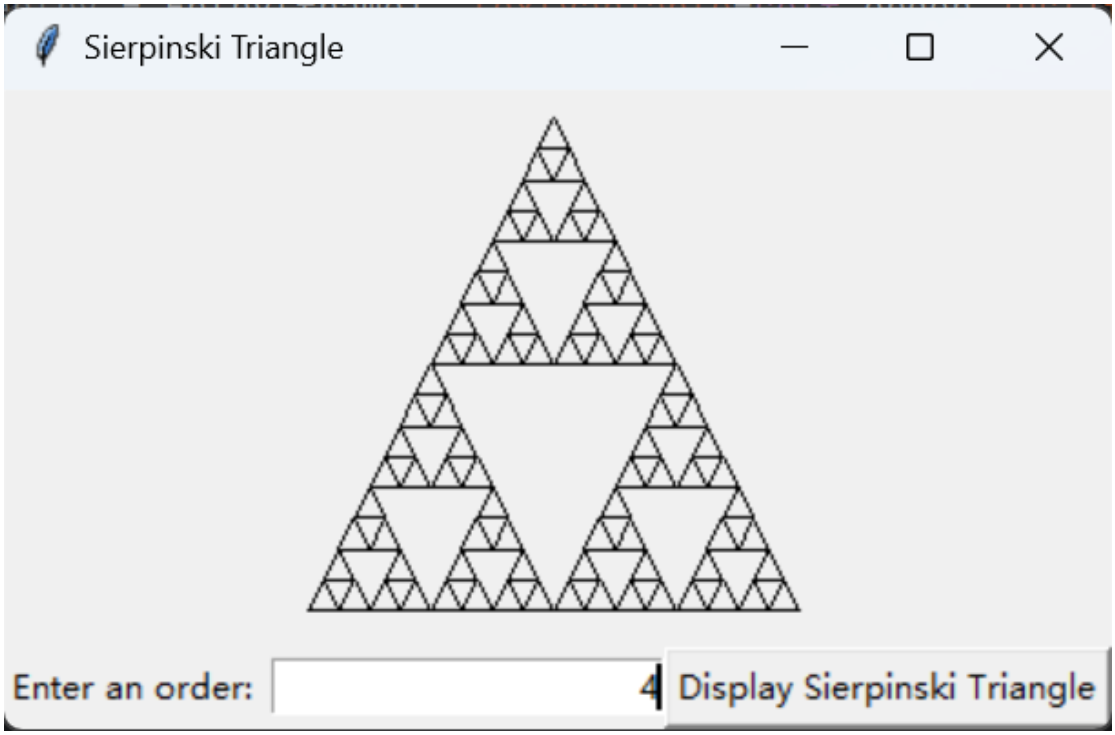
Orders tested: order = 4, 5, 6

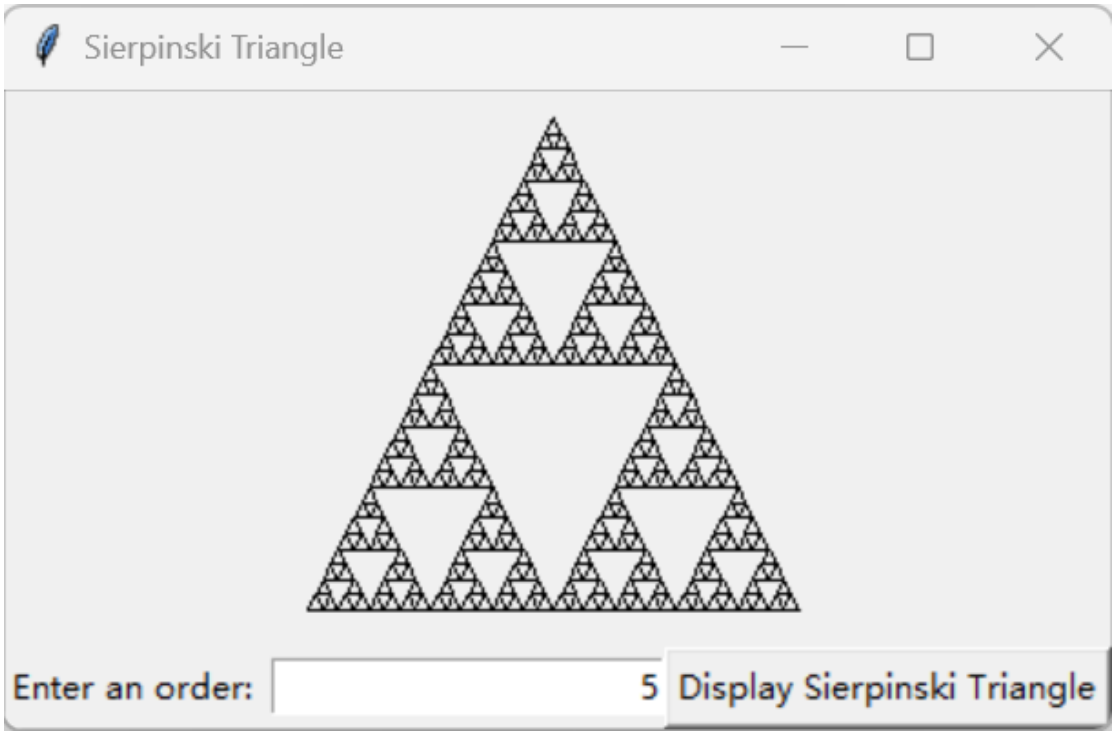
Screenshot descriptions

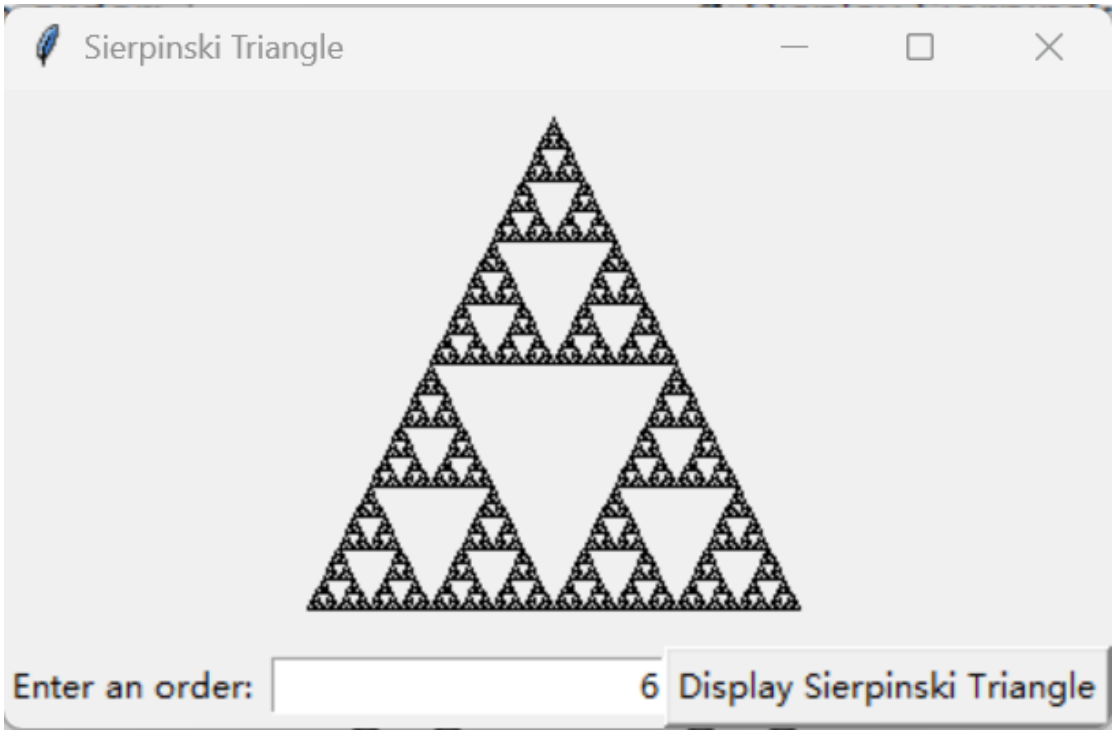
order = 4: The triangle is subdivided into 3^4 = 81 small triangles with hollow centers, presenting a fine fractal pattern.

order = 5: Further subdivision makes the self-similar structure more apparent; many small triangles appear at the edges.

order = 6: Due to high recursion depth, drawing takes longer, but eventually a very complex grid of hollow triangles is displayed.







4.2 Koch Snowflake

Program: `task4_koch_snowflake.py` (using Tkinter, modified from Sierpiński)

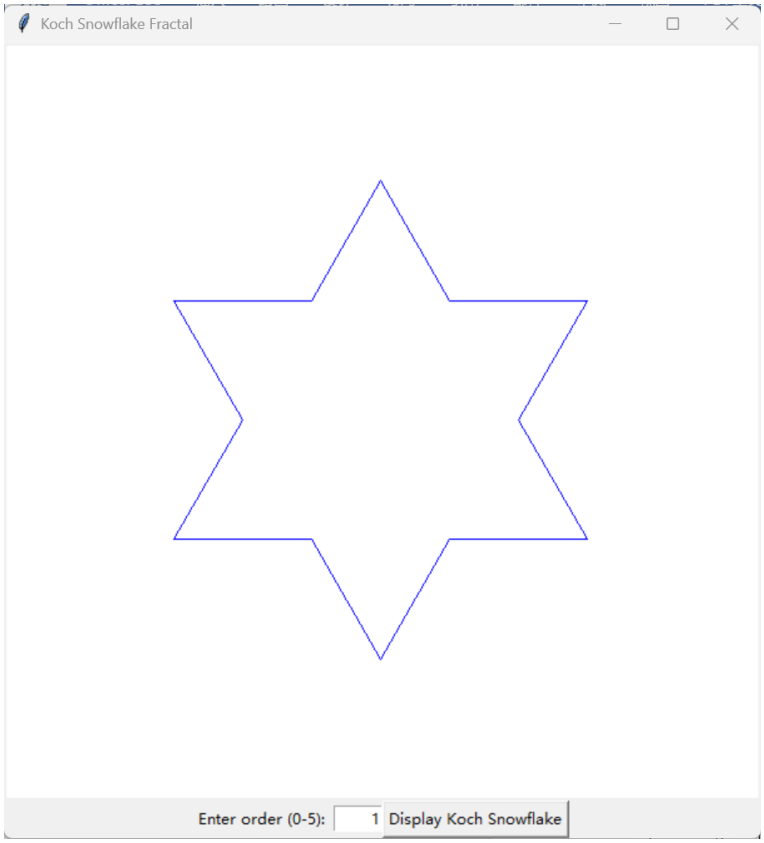
Orders tested: order = 1, 2, 3

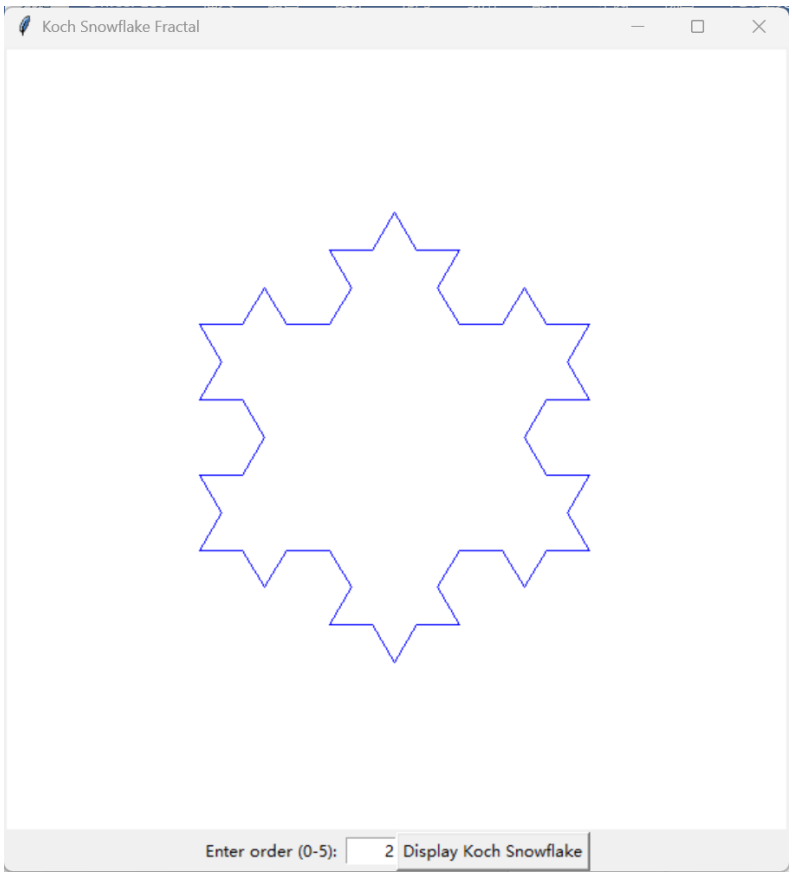
Screenshot descriptions

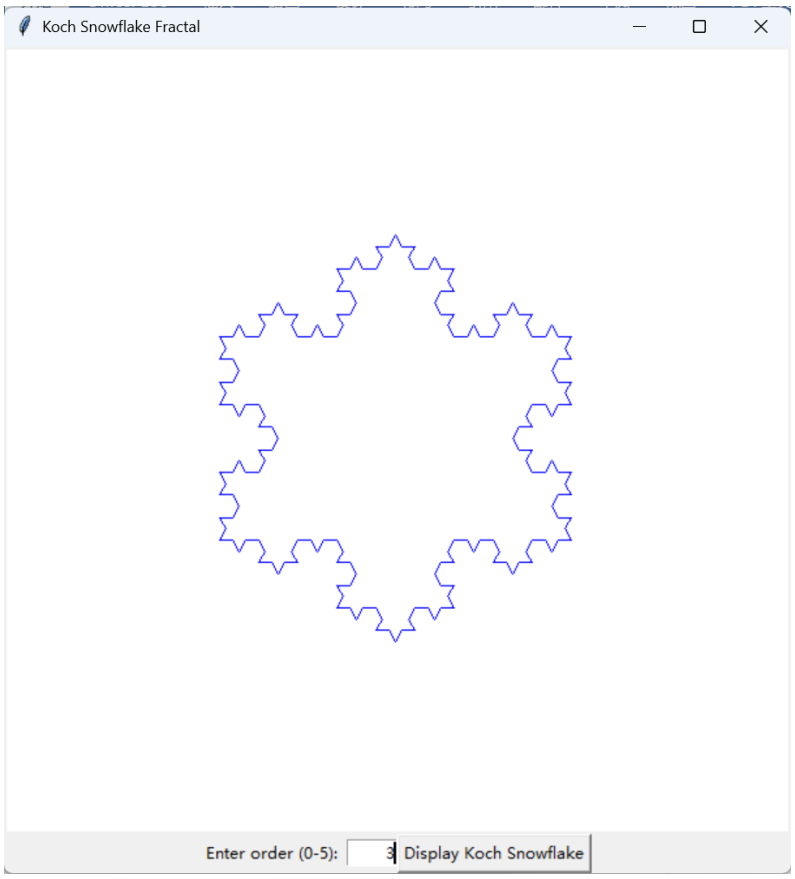
order = 1: Each side of an equilateral triangle has a smaller equilateral triangle protruding from its middle, forming a six-pointed star shape.

order = 2: Each line segment is further subdivided, with more small triangles protruding; the snowflake's edges become curved and complex.

order = 3: The classic Koch snowflake shape emerges, with an infinitely detailed boundary but finite area.







Fractal Analysis

Recursion is ideal for describing self-similar shapes: every small part has the same shape as the whole, only the scale changes.

As the order increases, the complexity of the figure grows exponentially, but the code size remains the same, demonstrating the conciseness of recursion.

The Sierpiński triangle is implemented by midpoint subdivision; the Koch snowflake is implemented by dividing each line segment into three equal parts and constructing an outward equilateral triangle.

Conclusion

Algorithm/Data Structure Selection Guide

Application scenario	Recommended approach	Reason
Simple repeated computation, controllable recursion depth	Recursion	Clean code, easy to understand
Large-scale computation or deep recursion	Iteration	Better performance, no stack overflow risk
Sorting small datasets (< 100)	Insertion sort	Small constant factor, simple implementation
Sorting large datasets	Merge sort / TimSort	O(n log n) efficiency
Problems requiring explicit step-by-step moves (e.g., Tower of Hanoi)	Recursion	Naturally matches problem decomposition
Drawing complex fractals	Recursion	Extremely short code, perfectly captures self-similarity

Performance Summary

Recursion: elegant code, but suffers from function call overhead and stack depth limits; recursive Fibonacci has extremely poor performance.

Iteration: better performance, safer, suitable for production environments.

Merge sort: far outperforms insertion sort on large datasets (25× faster for 5000 elements).

Fractal recursion: although computationally intensive, works well for human-perceptible

orders (≤6), with impressive visual results.

Learning Gains

The “base case + recursive case” concept in recursion and its connection to mathematical induction

The actual differences in time and space between recursion and iteration

The divide-and-conquer strategy of merge sort and its performance contrast with insertion sort

How to use recursion to solve classic problems (Tower of Hanoi) and extend functionality with counters

How to combine Tkinter graphics with recursion to draw fractal patterns

The geometric construction of the Koch snowflake and its recursive implementation details